

Sistemi Context-Aware, a.a. 2025-2026

Proposte di Progetti

Prof. Marco Di Felice, Prof. Angelo Trotta, Dr. Leonardo Ciabattini
{marco.difelice3, a.trotta, leonardo.ciabattini}@unibo.it

May 20, 2026

1 Proposta 1 – Context-Aware Campus Assistant

Focus: location awareness, geo-fencing, recommendation, spatial data management

Descrizione generale. Si chiede di realizzare una piattaforma context-aware pensata per supportare gli studenti durante la vita universitaria. Il sistema deve suggerire informazioni e servizi utili in base al contesto corrente dell'utente, considerando ad esempio posizione, orario, vicinanza a edifici universitari, servizi disponibili e preferenze personali.

L'applicazione deve essere in grado di mostrare su mappa luoghi rilevanti per lo studente, come aule, biblioteche, sale studio, mense, uffici e altri servizi universitari. Inoltre, deve generare suggerimenti o notifiche contestuali, ad esempio quando l'utente si trova vicino a un edificio rilevante oppure quando un servizio utile è disponibile nelle vicinanze.

Il progetto non deve limitarsi a mostrare una mappa statica. La parte context-aware deve essere esplicita: il sistema deve usare almeno due informazioni di contesto, ad esempio posizione e orario, per produrre un suggerimento diverso a seconda della situazione. Ad esempio, la stessa posizione potrebbe generare un suggerimento diverso al mattino, durante la pausa pranzo o a fine giornata.

- *App mobile.* L'app deve permettere:
 1. Visualizzazione su mappa dei punti di interesse universitari, e.g. aule, biblioteche, sale studio, mense, uffici, segreterie, fermate vicine. I punti di interesse devono essere organizzati in categorie e devono poter essere selezionati dall'utente per visualizzarne i dettagli.
 2. Rilevamento della posizione corrente dell'utente, previa gestione dei permessi di localizzazione. Se non si vuole usare una localizzazione

reale, è accettabile simulare la posizione dell'utente, purché la simulazione sia chiara e modificabile durante la demo.

3. Visualizzazione dei servizi vicini alla posizione dell'utente, ordinati per "rilevanza". La nozione di rilevanza deve essere spiegata nella relazione, anche se basata su una semplice regola, e.g. distanza minima, categoria preferita, servizio aperto in quel momento.
 4. Definizione di un profilo utente minimale, e.g. preferenze su servizi, campus di interesse, mezzi di spostamento preferiti. Il profilo non deve essere complesso, ma deve influenzare almeno una decisione del sistema.
 5. Ricezione di notifiche contestuali basate su posizione e orario, ad esempio "sei vicino a una biblioteca aperta" oppure "la mensa più vicina è a 300 metri". Le notifiche possono essere implementate come notifiche native, messaggi nell'app o eventi visualizzati nella dashboard.
 6. Storico personale delle notifiche o dei suggerimenti ricevuti, in modo da poter mostrare nella demo quali decisioni sono state prese dal sistema e in quale contesto.
- *Back-end.* Il back-end deve:
 1. Gestire un database spaziale contenente i punti di interesse universitari. Ogni elemento deve includere almeno: identificativo, nome, categoria, descrizione, coordinate/geometria, orari o metadati rilevanti. I dati possono essere reali, semplificati o inseriti manualmente.
 2. Fornire API REST per interrogare i punti di interesse vicini a una posizione. Ad esempio, data una coppia latitudine/longitudine e un raggio di ricerca, il sistema deve restituire i servizi presenti nell'area.
 3. Fornire API REST per filtrare i punti di interesse in base a categoria, distanza, orario di apertura o preferenze utente. I filtri devono essere effettivamente usati dall'app o dalla dashboard.
 4. Memorizzare eventi contestuali generati dal sistema, e.g. suggerimenti mostrati, notifiche inviate, PoI selezionati. Ogni evento dovrebbe includere almeno timestamp, utente o sessione, posizione approssimata e motivo del suggerimento.
 5. Implementare una logica base di ranking dei suggerimenti basata su almeno due fattori contestuali, ad esempio distanza e orario. Non è necessario usare machine learning: sono accettabili regole esplicite, purché siano motivate e dimostrate.
 - *Front-end.* La dashboard Web deve:
 1. Consentire la gestione CRUD dei punti di interesse, oppure almeno l'inserimento/modifica di un sottoinsieme significativo di essi.

2. Mostrare una mappa interattiva con i punti di interesse divisi per categoria. La mappa deve permettere di capire visivamente dove si trovano i servizi e quali sono rilevanti per l'utente corrente.
3. Visualizzare statistiche aggregate sull'uso del sistema, e.g. PoI più consultati, categorie più richieste, numero di suggerimenti generati.
4. Consentire il filtro per categoria, area geografica e intervallo temporale, così da analizzare diversi scenari di utilizzo.
5. Mostrare, almeno in forma testuale o tabellare, il motivo per cui un suggerimento è stato generato, e.g. "servizio vicino", "servizio aperto", "categoria preferita".

1.1 Componenti

- **Base 18 pt** Applicazione base:
 - App mobile con visualizzazione della posizione utente e dei punti di interesse universitari su mappa. La posizione può essere reale o simulata, ma deve poter cambiare per mostrare reazioni diverse del sistema.
 - Back-end con database spaziale dei PoI. Il database deve contenere un numero sufficiente di elementi per rendere significativa la ricerca, ad esempio almeno alcune categorie diverse e più punti per categoria.
 - API REST per interrogazione dei PoI vicini. Le API devono essere documentate nella relazione, indicando parametri di input e formato della risposta.
 - Dashboard Web minimale per gestione e visualizzazione dei PoI.
 - Meccanismo base di suggerimento contestuale basato su posizione e orario. Il suggerimento deve essere calcolato dal sistema e non scelto manualmente durante la demo.
- **+2 pt** Geo-fencing e notifiche contestuali:
 - Definizione di aree geografiche associate a edifici o servizi universitari. Le aree possono essere cerchi o poligoni.
 - Rilevamento di eventi di **entry/exit** da geofence, cioè ingresso o uscita dell'utente da un'area rilevante.
 - Invio di notifiche contestuali quando l'utente entra o esce da un'area rilevante. Deve essere chiaro quale regola ha generato la notifica.
- **+2 pt** Personalizzazione dei suggerimenti:
 - Profilo utente con preferenze configurabili. Ad esempio, lo studente può indicare che è interessato a sale studio e biblioteche ma non alle mense.

- Ranking dei PoI basato su preferenze, distanza, orario e categoria. Il ranking deve produrre risultati diversi per utenti o preferenze diverse.
- Storico dei suggerimenti ricevuti e possibilità di feedback dell'utente, e.g. utile/non utile, salvato, ignorato.
- **+2 pt** Integrazione con calendario o agenda:
 - Possibilità di inserire manualmente un calendario di lezioni/eventi. Non è necessario integrare Google Calendar o servizi esterni: è sufficiente una semplice agenda interna.
 - Suggerimenti contestuali rispetto al prossimo impegno, e.g. percorso verso l'aula, servizi vicini prima della lezione, avviso se l'utente è lontano dall'aula poco prima dell'inizio.
- **+2 pt** Analytics spaziale:
 - Heatmap dei punti di interesse più consultati o delle aree in cui vengono generati più suggerimenti.
 - Analisi delle aree più frequentate o più rilevanti. L'analisi può essere semplice, ma deve essere basata sui dati raccolti dal sistema.
 - Grafici temporali sull'utilizzo dei servizi, ad esempio numero di suggerimenti per ora o categorie più richieste durante la giornata.
- **+2 pt** Privacy location-aware:
 - Integrazione di almeno una tecnica di protezione della privacy della posizione tra quelle discusse a lezione, e.g. perturbazione, cloaking, dummy location.
 - Valutazione dell'impatto della tecnica su *Privacy Perturbation* e *Quality of Service*, mediante grafici. Ad esempio, si può mostrare come l'aumento della perturbazione migliori la privacy ma riduca la precisione dei suggerimenti.
- **+2 pt** Supporto offline:
 - Caching locale dei PoI in una determinata area, ad esempio i servizi del campus selezionato.
 - Visualizzazione della mappa e dei servizi salvati anche in assenza di connessione. Durante la demo deve essere mostrabile almeno un caso in cui l'app funziona con dati precedentemente salvati.
- **+2 pt** Containerizzazione con Docker Compose:
 - Containerizzazione di back-end, database e front-end.
 - Script Docker Compose per avviare l'intera piattaforma con un unico comando o con pochi comandi chiaramente documentati.

1.2 Tecnologie consigliate

- **App mobile:** Android nativo, Flutter, React Native o framework a scelta. *L'interfaccia dell'app può essere minimale e non è oggetto di valutazione.*
- **Back-end:** Node.js, Python, Java o framework a scelta.
- **Database:** Postgres/PostGIS per la gestione dei dati spaziali. In alternativa, per prototipi semplici, è possibile usare un database non spaziale o file GeoJSON, purché le operazioni spaziali richieste siano implementate chiaramente.
- **Front-end:** OpenLayers o Leaflet per la mappa; librerie a scelta per grafici.
- **Containerizzazione:** Docker e Docker Compose.

E' possibile utilizzare un qualsiasi linguaggio di programmazione o librerie a scelta dello studente.

2 Proposta 2 – Privacy-Aware Mobility Diary con HAR

Focus: human activity recognition, location awareness, privacy, mobility analytics

Descrizione generale. Si chiede di realizzare una piattaforma context-aware per la costruzione di un diario personale della mobilità. Il sistema deve raccogliere o simulare dati di posizione e dati sensoriali, ricostruire gli spostamenti dell'utente e produrre un diario giornaliero contenente informazioni significative, come luoghi visitati, tempi di permanenza, percorsi e modalità di spostamento.

A differenza di un semplice tracciamento GPS, il sistema deve includere una componente di Human Activity Recognition (HAR), in grado di riconoscere o stimare la modalità di mobilità dell'utente, ad esempio **walking, running, biking, driving, idle**. Il diario deve inoltre prevedere meccanismi per proteggere la privacy dei dati di posizione, distinguendo tra una vista privata dettagliata e una vista condivisibile/aggregata.

Un esempio di output atteso è una sequenza di eventi del tipo: “08:15–08:35, spostamento da zona A a zona B, modalità prevalente: bike”, “08:35–12:10, permanenza in zona universitaria”, “12:10–12:25, spostamento a piedi verso mensa”. Il sistema deve quindi trasformare dati grezzi, come punti GPS e misure sensoriali, in informazioni di contesto più leggibili e utili.

- *App mobile.* L'app deve permettere:
 1. Raccolta della posizione GPS dell'utente, oppure caricamento di tracce pre-registrate/simulate. Se si usano dati simulati, deve essere possibile mostrare almeno due giornate o due percorsi differenti.
 2. Raccolta di dati sensoriali utili per HAR, e.g. accelerometro, giroscopio, Activity Recognition API, oppure dataset esterni. Non è obbligatorio raccogliere dati reali da smartphone, ma il progetto deve includere una componente HAR effettiva o chiaramente simulata e motivata.
 3. Visualizzazione della traccia giornaliera su mappa, con distinzione tra segmenti di movimento e soste.
 4. Visualizzazione del diario della mobilità, con segmenti temporali del tipo: luogo visitato, durata, percorso, modalità di spostamento. Ogni segmento deve essere associato a un intervallo temporale e a una label di attività/mobilità.
 5. Visualizzazione di statistiche personali, e.g. tempo camminando, tempo in bici, tempo fermo, distanza percorsa, numero di luoghi visitati.
 6. Configurazione del livello di privacy desiderato per la condivisione o esportazione del diario, ad esempio scegliendo tra vista precisa, vista approssimata e vista aggregata.

- *Back-end.* Il back-end deve:
 1. Memorizzare tracce GPS, timestamp, segmenti di movimento e attività riconosciute. I dati devono essere organizzati in modo da poter ricostruire il diario anche dopo la raccolta o il caricamento.
 2. Gestire un modello dati per `MobilityTrace`, `MobilitySegment`, `SignificantPlace`, `ActivityLabel`. Nella relazione deve essere spiegato il significato di ciascuna entità.
 3. Fornire API REST per upload/download delle tracce e dei segmenti del diario. Ad esempio, deve essere possibile caricare una traccia e ottenere in risposta i segmenti riconosciuti.
 4. Implementare una logica per individuare luoghi significativi, e.g. permanenza superiore a una certa soglia temporale in una determinata area. La soglia scelta deve essere indicata e motivata.
 5. Implementare una logica base di segmentazione del percorso in base a cambiamenti di posizione, velocità o attività riconosciuta. Ad esempio, un cambio da `walking` a `bus` dovrebbe generare due segmenti distinti.
 6. Generare una versione privacy-aware del diario, con dati aggregati o perturbati. Deve essere possibile confrontare la versione privata e quella condivisibile.
- *Front-end.* La dashboard Web deve:
 1. Mostrare su mappa le tracce utente e i luoghi significativi.
 2. Visualizzare il diario giornaliero in forma tabellare e temporale, in modo che sia chiaro cosa ha fatto l'utente durante la giornata.
 3. Visualizzare statistiche aggregate sulle attività riconosciute, ad esempio distribuzione del tempo tra cammino, bici, auto, bus e sosta.
 4. Consentire il confronto tra traccia reale e traccia anonimizzata/perturbata. Il confronto deve rendere evidente l'effetto della tecnica di privacy scelta.
 5. Consentire il filtro per giorno, attività, luogo significativo o intervallo temporale.

2.1 Componenti

- **Base 18 pt** Applicazione base:
 - Raccolta, caricamento o simulazione di tracce GPS. La traccia deve includere almeno timestamp e coordinate.
 - Visualizzazione della mobilità giornaliera su mappa, con possibilità di distinguere visivamente le diverse parti della giornata.

- Segmentazione base della traccia in spostamenti e soste. Le regole di segmentazione possono essere semplici, ma devono essere automatiche e documentate.
 - Diario della mobilità con luoghi significativi e intervalli temporali. Il diario deve essere più informativo della semplice lista di punti GPS.
 - Integrazione HAR base tramite API di sistema o classificazione semplice delle modalità di movimento. Ad esempio, è possibile usare velocità media e accelerazione per distinguere tra fermo, cammino e veicolo, oppure usare un dataset HAR e un classificatore.
 - Dashboard Web per visualizzazione di tracce, luoghi significativi e statistiche.
- **+3 pt** HAR avanzato e riconoscimento multimodale della mobilità:
 - Implementazione di un modulo di riconoscimento della modalità di movimento dell'utente, distinguendo almeno tre classi, e.g. **walking**, **biking**, **driving**, **idle**. Il dataset usato, reale o simulato, deve essere descritto nella relazione.
 - Il riconoscimento deve basarsi su dati sensoriali, dati GPS o una combinazione dei due. Ad esempio, è possibile usare accelerometro, giroscopio, velocità GPS, variazioni di posizione, pattern temporali e/o rilevamento delle fermate.
 - Se viene usato un modello di Machine Learning, devono essere descritte le feature estratte dai dati sensoriali, e.g. media, varianza, energia, minimo/massimo, finestre temporali. La scelta delle feature deve essere motivata.
 - Il sistema deve gestire e discutere almeno alcuni casi ambigui, ad esempio traffico lento in auto, cammino veloce, fermate temporanee o cambio di mezzo di trasporto.
 - Le prestazioni del riconoscimento devono essere valutate mediante metriche adeguate, e.g. accuracy, precision, recall e confusion matrix. I risultati devono essere riportati nella relazione e non solo mostrati nel codice.
 - **+2 pt** Route assistant context-aware:
 - Possibilità per l'utente di selezionare un punto A e un punto B.
 - Suggerimento del percorso in base alla modalità di mobilità riconosciuta o selezionata. Ad esempio, percorso pedonale se l'utente cammina, percorso ciclabile se l'utente è in bici, percorso stradale se è in auto.
 - Aggiornamento dinamico del percorso se cambia la modalità di mobilità, e.g. da **walking** a **bus**. L'aggiornamento può essere simulato, ma deve essere dimostrabile.

- **+2 pt** Riconoscimento avanzato di luoghi significativi:
 - Clustering dei punti GPS per individuare aree frequentemente visitate o soste ricorrenti.
 - Classificazione dei luoghi in categorie, e.g. casa, università, lavoro, palestra, altro, anche mediante etichette manuali.
 - Associazione dei luoghi significativi ai segmenti del diario, in modo da produrre descrizioni più leggibili.
- **+3 pt** Privacy dei dati di posizione:
 - Integrazione di almeno una tecnica di privacy presentata a lezione, e.g. perturbazione, cloaking spaziale, aggregazione temporale, dummy location.
 - Generazione di due viste del diario: una privata dettagliata e una condivisibile privacy-aware.
 - Studio della metrica di *Privacy Perturbation*, intesa come distanza tra posizione reale e posizione pubblicata.
 - Studio della *Quality of Service*, intesa come perdita di accuratezza nella ricostruzione del diario o delle statistiche. Ad esempio, si può misurare quanti luoghi significativi vengono ancora riconosciuti dopo la perturbazione.
 - Visualizzazione mediante grafici del trade-off tra privacy e qualità del servizio.
- **+2 pt** Analytics personali:
 - Grafici giornalieri o settimanali su tempo trascorso in movimento, fermo, in bici, in auto, a piedi.
 - Heatmap dei luoghi più frequentati.
 - Statistiche sui percorsi più frequenti, sulle distanze percorse e sulle modalità di mobilità prevalenti.
- **+2 pt** Containerizzazione con Docker Compose:
 - Containerizzazione di back-end, database e front-end.
 - Script Docker Compose per avviare l'intera piattaforma con un unico comando o con pochi comandi chiaramente documentati.

2.2 Tecnologie consigliate

- **App mobile:** Android nativo, Flutter, React Native o framework a scelta. *L'interfaccia dell'app può essere minimale e non è oggetto di valutazione.*

- **HAR:** Activity Recognition API del sistema operativo, sensori Android/iOS, oppure dataset pubblici di HAR/mobility recognition. In caso di dataset pubblico, è necessario spiegare come le classi del dataset sono state adattate al progetto.
- **Machine Learning:** Python, scikit-learn, TensorFlow/Keras o librerie equivalenti.
- **Back-end:** Node.js, Python, Java o framework a scelta.
- **Database:** Postgres/PostGIS per dati spaziali; database a scelta per dati temporali.
- **Front-end:** OpenLayers o Leaflet per la mappa; librerie a scelta per grafici.
- **Containerizzazione:** Docker e Docker Compose.

E' possibile utilizzare un qualsiasi linguaggio di programmazione o librerie a scelta dello studente.

3 Proposta 3 – Context-Aware Edge Deployment per Smart Event Crowd Monitoring

Focus: edge computing, Kubernetes, context-aware deployment, location awareness

Descrizione generale. Si chiede di realizzare una piattaforma context-aware per il monitoraggio della densità di persone durante un evento pubblico, ad esempio una fiera, un concerto, una manifestazione sportiva, un evento universitario o una festa cittadina. Il sistema deve raccogliere dati, reali o simulati, provenienti da diverse aree dell'evento e stimare il livello di affollamento in ciascuna zona.

La caratteristica principale del progetto è l'integrazione con un'infrastruttura edge/cloud. I servizi di analisi devono poter essere eseguiti su nodi differenti, ad esempio edge node vicino all'ingresso principale, edge node vicino a un palco, edge node in un'altra zona dell'evento, oppure cloud. Il sistema deve decidere dove eseguire o instradare il servizio di analisi in base al contesto corrente, ad esempio densità della folla, posizione della zona monitorata, carico dei nodi, latenza stimata, priorità dell'area o disponibilità delle risorse.

Il progetto può essere interamente simulato: non è necessario disporre di sensori reali o di un evento reale. Tuttavia, la simulazione deve rappresentare uno scenario credibile, con più aree dell'evento, valori di affollamento che cambiano nel tempo e almeno una decisione context-aware relativa al posizionamento o all'instradamento del servizio.

- *Sorgenti dati.* Il sistema deve prevedere una o più sorgenti dati:
 1. Simulatori software di sensori o utenti distribuiti nelle aree dell'evento. Ad esempio, uno script può generare periodicamente misure diverse per ingresso, palco, area ristoro e uscita.
 2. Dati manualmente inseriti da utenti/operatori, e.g. segnalazioni di affollamento. Questa opzione è utile per mostrare nella demo come il sistema reagisce a un evento critico.
 3. Dataset esterni o tracce simulate. In questo caso, il dataset deve essere adattato allo scenario dell'evento e descritto nella relazione.
 4. Ogni dato deve includere almeno: identificativo area, timestamp, posizione o geofence, valore di crowd density o numero stimato di persone. Se si usano dati sintetici, deve essere spiegato come sono stati generati.
- *Back-end.* Il back-end deve:
 1. Gestire la definizione delle aree dell'evento, e.g. ingressi, uscite, palco, stand, corridoi, aree ristoro. Ogni area deve avere un identificativo, una posizione e possibilmente una priorità.

2. Memorizzare misure di affollamento associate a ciascuna area, mantenendo anche lo storico temporale necessario per grafici e analisi.
 3. Fornire API REST per l'invio e la lettura dei dati di crowd monitoring. Le API devono permettere sia l'inserimento di nuove misure sia la lettura dello stato corrente dell'evento.
 4. Calcolare lo stato corrente di ciascuna area, e.g. **low, medium, high, critical**. Le soglie devono essere configurabili o almeno dichiarate chiaramente.
 5. Generare alert quando una zona supera una soglia configurabile. L'alert deve includere area, timestamp, livello di gravità e possibile causa.
 6. Esporre informazioni sul contesto del sistema, e.g. area monitorata, priorità, carico del nodo, latenza stimata. Queste informazioni devono essere usate per prendere decisioni di deployment o routing.
- *Servizi edge*. Il progetto deve prevedere almeno un servizio containerizzato di analisi:
 1. Il servizio riceve dati relativi a una o più aree dell'evento.
 2. Il servizio calcola indicatori di affollamento, trend temporali o alert. Ad esempio, può calcolare la media degli ultimi minuti o rilevare un aumento improvviso.
 3. Il servizio deve poter essere deployato su nodi differenti, simulando edge node e cloud node. I nodi possono essere macchine reali, nodi minikube, container distinti o ambienti simulati.
 4. I nodi devono essere etichettati in base alla loro posizione logica, e.g. **area=entrance, area=stage, tier=edge, tier=cloud**. Le etichette devono essere usate per guidare o spiegare il placement.
 5. Deve essere chiaro quale decisione context-aware viene presa: ad esempio, spostare il servizio vicino all'area critica, inviare richieste al nodo meno carico, oppure usare il cloud quando gli edge node non sono disponibili.
 - *Front-end*. La dashboard Web deve:
 1. Mostrare una mappa dell'evento con le diverse aree monitorate.
 2. Visualizzare il livello corrente di crowd density per ciascuna area, possibilmente con colori, icone o etichette diverse.
 3. Mostrare alert e storico degli eventi critici.
 4. Visualizzare su quale nodo edge/cloud è attivo il servizio di analisi o verso quale nodo vengono instradate le richieste.
 5. Mostrare metriche di base, e.g. numero di richieste, latenza media simulata, carico del nodo, numero di alert.

6. Permettere di cambiare almeno un parametro dello scenario, e.g. aumentare manualmente l'affollamento di una zona, modificare una soglia o simulare il sovraccarico di un nodo.

3.1 Componenti

- **Base 18 pt** Applicazione base:
 - Definizione delle aree dell'evento su mappa. Devono esserci almeno alcune aree distinte, con ruoli diversi, e.g. ingresso, palco, stand, uscita.
 - Simulazione o raccolta di dati di crowd density. I dati devono cambiare nel tempo per permettere al sistema di reagire.
 - Back-end con API REST per dati, aree e alert.
 - Dashboard Web per visualizzare mappa, livelli di affollamento e alert.
 - Containerizzazione del servizio di analisi. Il servizio deve essere separato logicamente dal resto dell'applicazione.
 - Deployment base del servizio mediante Docker o Kubernetes. Per la parte base è sufficiente mostrare che il servizio può essere eseguito come container.
 - Una decisione context-aware minimale, ad esempio selezionare il nodo o il servizio da usare in base all'area più affollata o alla latenza stimata.
- **+2 pt** Modellazione spaziale avanzata dell'evento:
 - Definizione delle aree come poligoni/geofence su mappa, invece di semplici punti.
 - Query spaziali per associare automaticamente una misura o una segnalazione all'area corrispondente. Ad esempio, una segnalazione con coordinate GPS deve essere associata al poligono che la contiene.
 - Visualizzazione differenziata delle aree in base al livello di affollamento.
- **+2 pt** Analytics e predizione del crowding:
 - Calcolo di trend temporali per ciascuna area, e.g. affollamento medio, massimo, variazione nel tempo.
 - Predizione semplice del livello di affollamento futuro, e.g. media mobile, regressione o modello ML. Non è necessario un modello complesso, ma deve essere spiegato perché è utile nello scenario.
 - Grafici temporali per confrontare aree diverse e supportare le decisioni del sistema.
- **+2 pt** Alert context-aware:

- Generazione di alert con priorità diversa in base a crowd density, durata della congestione e tipo di area. Ad esempio, un affollamento elevato vicino a un'uscita può essere più critico dello stesso valore in un'area aperta.
 - Notifiche diverse per utenti, organizzatori o operatori. Le notifiche possono essere simulate nella dashboard.
 - Suggerimenti di azioni contestuali, e.g. “indirizzare gli utenti verso ingresso B”, “aprire un varco secondario”, “spostare operatori verso area palco”.
- **+3 pt** Kubernetes context-aware deployment:
 - Deployment dei servizi su un cluster Kubernetes locale, e.g. minikube multi-node.
 - Uso di node labels per rappresentare edge node e cloud node. Ad esempio: `tier=edge`, `area=stage`, `area=entrance`, `tier=cloud`.
 - Uso di Node Affinity, Anti-Affinity, Taints/Tolerations o Topology Spread Constraints per controllare il placement dei servizi.
 - Dimostrazione sperimentale di almeno due politiche di placement. Ad esempio, una politica cloud-only e una politica edge-aware, oppure una politica vicino all'area critica e una bilanciata sul carico.
 - **+3 pt** Scheduler o orchestrazione context-aware:
 - Implementazione di una logica che decide il nodo migliore in base al contesto, e.g. area più affollata, latenza, carico, disponibilità.
 - Possibile implementazione mediante custom scheduler semplificato, controller esterno o script di orchestration. Non è necessario implementare uno scheduler Kubernetes completo: è sufficiente una logica esterna che osserva il contesto e modifica il deployment o il routing.
 - Confronto tra deployment statico e deployment context-aware, mostrando almeno un caso in cui la decisione cambia a causa del contesto.
 - **+2 pt** Failover e cloud fallback:
 - Gestione del caso in cui un edge node sia non disponibile o sovraccarico. Il guasto o sovraccarico può essere simulato.
 - Reindirizzamento verso un altro edge node o verso il cloud.
 - Visualizzazione nella dashboard del nodo attualmente utilizzato e del motivo del fallback.
 - **+2 pt** Valutazione sperimentale:
 - Misura o simulazione di metriche come latenza, carico dei nodi, numero di alert gestiti, tempo di risposta.

- Confronto tra almeno due configurazioni, e.g. cloud-only vs edge-aware, deployment statico vs context-aware, edge vicino vs edge remoto.
- Grafici e discussione dei risultati nella relazione. La valutazione deve essere collegata alle scelte progettuali e non limitarsi a mostrare screenshot.

3.2 Tecnologie consigliate

- **Back-end:** Node.js, Python, Java o framework a scelta.
- **Database:** Postgres/PostGIS per aree e dati spaziali; database a scelta per dati temporali.
- **Front-end:** OpenLayers o Leaflet per la mappa; librerie a scelta per grafici.
- **Containerizzazione:** Docker e Docker Compose.
- **Orchestrazione:** Kubernetes, minikube multi-node o piattaforme equivalenti.
- **Placement context-aware:** Node Affinity, Anti-Affinity, Taints/Tolerations, Topology Spread Constraints, custom scheduler semplificato o controller esterno.

E' possibile utilizzare un qualsiasi linguaggio di programmazione o librerie a scelta dello studente.

4 Proposta 4 – Student Urban Accessibility Advisor

Focus: spatial recommendation systems, urban analytics, open data integration, context-aware services

Descrizione generale. Si chiede di realizzare una piattaforma context-aware per supportare studenti universitari nell'analisi dell'accessibilità urbana e della qualità dei servizi disponibili nelle diverse aree della città. Il sistema deve integrare dati spaziali provenienti da fonti open data eterogenee, ad esempio:

- trasporto pubblico,
- sedi universitarie,
- biblioteche,
- mense,
- piste ciclabili,
- aree verdi,
- sale studio,
- servizi commerciali,
- mobilità sostenibile.

L'obiettivo della piattaforma è generare indicatori spaziali e suggerimenti relativi alle aree della città più adatte alla vita universitaria, sulla base delle preferenze dell'utente e del contesto corrente. Ad esempio:

- uno studente potrebbe preferire aree ben collegate tramite autobus;
- un altro potrebbe preferire zone ricche di biblioteche e sale studio;
- un altro ancora potrebbe privilegiare mobilità sostenibile e presenza di piste ciclabili.

Il sistema non deve limitarsi a visualizzare una mappa statica. La componente context-aware deve essere esplicita: il sistema deve utilizzare almeno due informazioni contestuali, ad esempio posizione, orario, densità dei servizi o preferenze utente, per generare suggerimenti differenti a seconda dello scenario.

Il progetto deve inoltre integrare:

- funzionalità GIS;
- query spaziali;
- ranking geografici;
- dashboard interattive;
- analisi spaziali dei servizi urbani.

4.1 Componenti

- **Base 18 pt** Applicazione base:
 - Integrazione di almeno 4 dataset open data differenti relativi alla città di Bologna.
 - Database spaziale per la gestione dei layer geografici tramite PostGIS.
 - Dashboard Web con mappa interattiva multilayer.
 - Visualizzazione su mappa dei dati spaziali:
 - * sedi universitarie,
 - * fermate TPER,
 - * biblioteche,
 - * mense,
 - * piste ciclabili,
 - * sale studio,
 - * aree verdi.
 - Sistema base di ranking delle aree urbane basato su almeno due fattori contestuali, e.g. distanza dai servizi e accessibilità tramite trasporto pubblico.
 - API REST per interrogazione dei servizi vicini a una posizione.
 - Possibilità di selezionare un'area della città e visualizzarne indicatori aggregati.
 - Dashboard Web per visualizzazione dei layer geografici e dei suggerimenti contestuali.
- **+2 pt** Profilazione utente e ranking personalizzato:
 - Definizione di preferenze configurabili da parte dello studente, e.g.:
 - * importanza del trasporto pubblico,
 - * preferenza per mobilità sostenibile,
 - * interesse per biblioteche o sale studio,
 - * preferenza per aree verdi.
 - Ranking dinamico delle aree urbane sulla base delle preferenze configurate.
 - Possibilità di confrontare ranking differenti per utenti diversi.
 - Storico dei suggerimenti generati dal sistema.
- **+2 pt** Analisi spaziale dei servizi urbani:
 - Implementazione di buffer analysis e density analysis sui servizi urbani.
 - Calcolo della densità dei punti di interesse nelle diverse aree della città.

- Heatmap relative alla distribuzione dei servizi universitari.
- Analisi delle aree maggiormente servite o maggiormente accessibili.
- **+2 pt** Mobility analytics e trasporto pubblico:
 - Integrazione di dati GTFS relativi al trasporto pubblico locale (TPER).
 - Calcolo del tempo stimato di percorrenza verso le sedi universitarie.
 - Analisi multimodale:
 - * walking,
 - * biking,
 - * trasporto pubblico.
 - Visualizzazione di isocrone temporali sulla mappa.
 - Suggerimento delle aree più facilmente raggiungibili tramite trasporto pubblico.
- **+2 pt** Recommendation engine context-aware:
 - Generazione automatica di suggerimenti sulle aree più adatte alla vita universitaria.
 - Calcolo di uno “Student Accessibility Score”.
 - Motivazione dei suggerimenti generati, e.g.:
 - * “area ben collegata”,
 - * “alta densità di biblioteche”,
 - * “presenza di piste ciclabili”.
 - Ranking dinamico delle zone urbane.
- **+2 pt** Temporal analytics:
 - Gestione di dati dipendenti dall’orario, esempio:
 - * orari biblioteche,
 - * disponibilità servizi,
 - * frequenza autobus.
 - Filtri temporali per fascia oraria o giorno della settimana.
 - Analisi differenziata tra scenari diurni/notturni.
- **+2 pt** Privacy location-aware:
 - Integrazione di almeno una tecnica di privacy della posizione tra quelle viste a lezione.
 - Possibilità di perturbare o anonimizzare la posizione utente.
 - Valutazione delle metriche di:
 - * Privacy Perturbation,

- * Quality of Service.
- Visualizzazione grafica del trade-off tra privacy e accuratezza dei suggerimenti.
- **+2 pt** Analytics avanzata e clustering:
 - Clustering delle aree urbane mediante algoritmi spaziali, e.g. K-Means o DBSCAN.
 - Individuazione automatica delle zone maggiormente accessibili.
 - Analisi di Moran per l'autocorrelazione spaziale di almeno 1 indicatore (es., densità delle biblioteche).
 - Visualizzazione dei cluster su mappa.
- **+2 pt** Containerizzazione con Docker Compose:
 - Containerizzazione di:
 - * database,
 - * back-end,
 - * dashboard Web.
 - Script Docker Compose per l'avvio dell'intera piattaforma.
 - Deployment opzionale mediante Kubernetes o minikube.

4.2 Tecnologie consigliate

- **App mobile opzionale:** Flutter, React Native, Android nativo o framework equivalente.
- **Back-end:** Python/FastAPI, Node.js o framework equivalente.
- **Database:** PostgreSQL + PostGIS.
- **Front-end:** OpenLayers o Leaflet.
- **Open data:**
 - OpenData Comune di Bologna,
 - GTFS TPER,
 - OpenStreetMap,
 - dataset derivati manualmente.
- **Containerizzazione:** Docker, Kubernetes.

È possibile utilizzare un qualsiasi linguaggio di programmazione o librerie a scelta dello studente.

5 Proposte Speciali

Le proposte di progetto seguenti sono valide per un singolo gruppo, e riguardano spunti di ricerca relativi ai sistemi context-aware:

- Migrazione dinamica di servizi software in scenari edge-cloud continuum, mediante tecnologia **WebAssembly**.
- Studio e valutazione di sistemi di riconoscimento delle attività umane per applicazioni bio-medicali.
- Studio e valutazione di sistemi di riconoscimento delle attività umane in ambienti distribuiti, mediante approcci di federated learning.
- Studio e valutazione di sistemi di riconoscimento delle attività su dispositivi embedded con limitate capacità computazionali, usando tecniche di edge AI.
- Studio e valutazione di sistemi di contact-tracing utilizzando dispositivi IoT per social analysis.
- Analisi e sviluppo di edge orchestration, servizi e dati, per nodi mobili, e.g. droni.

Le specifiche delle proposte sovra-elencate saranno fornite dal docente ai gruppi interessati.